



35. Python: Graphviz

Visualize Relationships. Understand Structures.

Previous Section:

[!\[\]\(c3d993ca47bfe2a953c700506ce31fa0_img.jpg\) 34. Python: Audio Processing](#)

Contents:

[1. Introduction to Graphviz](#)

[2. Visualization with Graphviz](#)

[2.1. Undirected and Directed Graphs](#)

[2.2. Trees](#)

[2.3. Coloring Graphs](#)

[2.4. Neural Networks](#)

[2.5. Saving the Graphs](#)

[Summary](#)

[References](#)

As programs become larger and data becomes more interconnected, understanding relationships between objects, functions, processes, and systems becomes increasingly difficult. A diagram is often worth far more than a page of explanations.

Graphviz is a powerful visualization library that transforms relationships into diagrams automatically. Instead of manually drawing boxes and arrows, we describe how objects are connected and Graphviz generates professional-quality visualizations for us.

Graphviz is widely used in: Software Engineering; Network Design; Database Modeling; Artificial Intelligence; Dependency Analysis; Workflow Visualization;

Organizational Charts

Whether you want to visualize a family tree, a computer network, a state machine, or a machine learning decision tree, Graphviz provides a simple and elegant solution.

1. Introduction to Graphviz

Graphviz is an open-source graph visualization toolkit developed by AT&T Research. Its primary purpose is to transform a collection of **nodes** and **relationships** into clear graphical representations.

At its core, Graphviz works with two fundamental concepts:

Component	Description
Node	An object or entity
Edge	A relationship between two nodes

For example:

- People connected through friendships
- Computers connected through networks
- Functions connected through calls
- Tasks connected through dependencies

These can all be represented as graphs.

Instead of manually positioning objects on a canvas, Graphviz automatically determines an effective layout and generates a clean diagram.

Why Use Graphviz?

Traditional drawing tools require manual placement of every object. As diagrams grow larger, maintaining them becomes difficult.

Graphviz solves this problem by automatically: Positioning nodes. Drawing connections. Organizing layouts. Reducing overlaps. Improving readability.

As a result, even graphs containing hundreds or thousands of nodes can remain understandable.

Directed and Undirected Graphs

Graphviz supports two major graph types.

- **Directed Graphs:** A directed graph contains relationships with a direction. Examples include:
 - Parent → Child;
 - Task A → Task B;
 - Function A → Function B;
 - Manager → Employee.

The direction indicates flow or dependency.

- **Undirected Graphs:** An undirected graph represents mutual relationships. Examples include:
 - Friendships
 - Social Connections
 - Computer Clusters
 - Team Memberships

In these graphs, the connection itself is important rather than the direction.

Graphviz Layout Engines

One of Graphviz's greatest strengths is its collection of layout engines. Different engines arrange nodes differently depending on the graph structure.

Engine	Best Used For
<code>dot</code>	Hierarchical diagrams and trees
<code>neato</code>	General network graphs
<code>fdp</code>	Force-directed graphs
<code>sfdp</code>	Large-scale network visualization
<code>circo</code>	Circular relationships
<code>twopi</code>	Radial diagrams

Choosing the right engine can dramatically improve readability.

Common Applications of Graphviz

Graphviz appears in many areas of computing.

- **Software Engineering:** Developers use Graphviz to visualize Function calls; Module dependencies; Class hierarchies; Project architectures
- **Database Systems:** Database designers use Graphviz to generate: Entity-Relationship Diagrams (ERD); Table relationships; Foreign key mappings
- **Computer Networks:** Network administrators use Graphviz to display Routers; Switches; Firewalls; Network topology
- **Artificial Intelligence:** Machine learning practitioners frequently visualize Decision Trees; Neural Network Structures; Bayesian Networks; Knowledge Graphs
- **Business and Management:** Organizations use Graphviz to create Organizational Charts; Workflow Pipelines; Process Diagrams; Project Dependencies

Graphviz Output Formats

Graphviz can export diagrams into multiple formats.

Format	Usage
PNG	General-purpose image
JPG	Compressed image
SVG	Scalable vector graphics
PDF	Documents and reports
DOT	Graph description language
JSON	Data exchange and integration

This flexibility allows Graphviz diagrams to be embedded into websites, reports, applications, and documentation.

Graphviz and Python

Python provides excellent integration with Graphviz through libraries such as:

Library	Purpose
<code>graphviz</code>	Direct Graphviz integration
<code>pydot</code>	DOT language manipulation
<code>networkx</code>	Graph analysis with Graphviz visualization
<code>sklearn.tree</code>	Decision tree visualization using Graphviz

These libraries allow Python programs to generate diagrams dynamically from data structures, databases, or algorithms.

As a result, Graphviz becomes more than a drawing tool—it becomes a way to transform data into visual knowledge automatically.

Most visualization libraries focus on plotting numbers. Graphviz focuses on visualizing **relationships**.

Instead of answering questions such as: ***How large is this value?*** Graphviz answers questions such as: ***What is connected to what? What depends on what? How does information flow through the system?***

This makes Graphviz one of the most valuable tools for understanding complex structures, making hidden relationships visible, and communicating system designs clearly.

2. Visualization with Graphviz

The true power of Graphviz comes from its ability to transform simple descriptions into meaningful visual diagrams.

Instead of manually drawing nodes and arrows, we simply describe the relationships and let Graphviz handle the layout automatically.

In this section, we will explore several common graph structures, from simple directed graphs to neural network architectures.

2.1. Undirected and Directed Graphs

The simplest graphs consist of nodes and connections.

A **directed graph** uses arrows to indicate the direction of a relationship.

```

from graphviz import Digraph

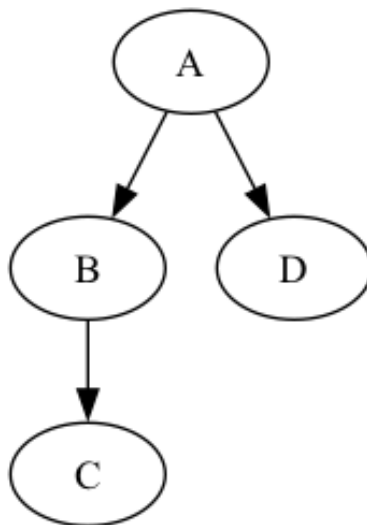
g = Digraph("Directed Graph", format="png")

g.edge("A", "B")
g.edge("B", "C")
g.edge("A", "D")

g.view()

```

The `format` parameter sets the file extension—like **PNG** or **PDF**—for your rendered graph image. Default is PDF



Such graphs are commonly used for: Workflows; Function calls; Dependencies; State transitions

An **undirected graph** represents relationships without direction.

```

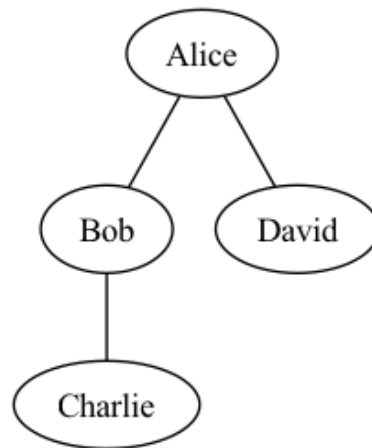
from graphviz import Graph

g = Graph("Undirected Graph", format="png")

g.edge("Alice", "Bob")
g.edge("Bob", "Charlie")
g.edge("Alice", "David")

```

```
g.view()
```



Undirected graphs are often used for: Social networks; Friendships; Communication networks; Computer clusters.

2.2. Trees

A tree is a special graph structure where each child has exactly one parent. Trees appear everywhere in Computer Science: File systems; Decision trees; XML documents; HTML DOM structures; Family trees

Graphviz makes tree visualization extremely simple.

```
from graphviz import Digraph

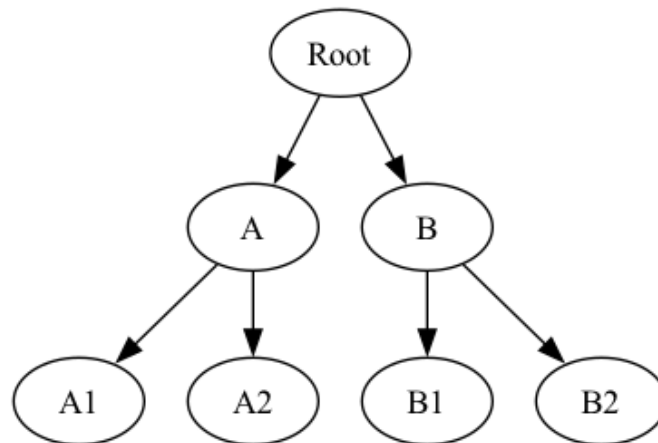
tree = Digraph(format="png")

tree.edge("Root", "A")
tree.edge("Root", "B")

tree.edge("A", "A1")
tree.edge("A", "A2")

tree.edge("B", "B1")
tree.edge("B", "B2")
```

```
tree.view()
```



Notice how Graphviz automatically arranges the hierarchy. This automatic layout becomes extremely valuable when the tree contains hundreds of nodes.

2.3. Coloring Graphs

Nodes and edges can be customized with colors, shapes, and styles.

Coloring helps emphasize different categories, roles, or states within a graph.

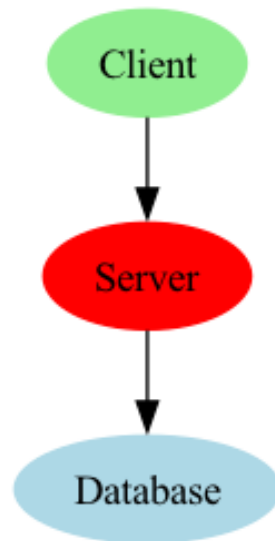
```
from graphviz import Digraph

g = Digraph(format="png")

g.node("Server", color="red", style="filled")
g.node("Database", color="lightblue", style="filled")
g.node("Client", color="lightgreen", style="filled")

g.edge("Client", "Server")
g.edge("Server", "Database")

g.view()
```

Common customization options include:

Property	Purpose
color	Border color
fillcolor	Interior color
style	Filled, dashed, bold
shape	Circle, box, diamond, ellipse
fontcolor	Text color

For example:

```
g.node("A", shape="diamond")
g.node("B", shape="box")
g.node("C", shape="circle")
```

Different shapes can make diagrams significantly easier to understand.

2.4. Neural Networks

Graphviz is frequently used to visualize Artificial Neural Networks. Neural networks consist of: Input Layer; Hidden Layer(s); Output Layer

The following Graphviz script generates a simple feed-forward neural network.

```

from graphviz import Digraph

nn = Digraph("Neural Network", format="png")
nn.attr(rankdir="LR", splines="line")
nn.attr("node", fixedsize="true", shape="circle")

# Input Layer
with nn.subgraph(name="cluster_input") as c:
    c.attr(color="white", label="Input Layer")
    c.attr("node", color="blue4")
    for node in ["x1", "x2", "x3", "x4"]:
        c.node(node)

# Hidden Layer
with nn.subgraph(name="cluster_hidden") as c:
    c.attr(color="white", label="Hidden Layer")
    c.attr("node", color="red2")
    for node in ["h1", "h2", "h3"]:
        c.node(node)

# Output Layer
with nn.subgraph(name="cluster_output") as c:
    c.attr(color="white", label="Output Layer")
    c.attr("node", color="seagreen2")
    c.node("0")

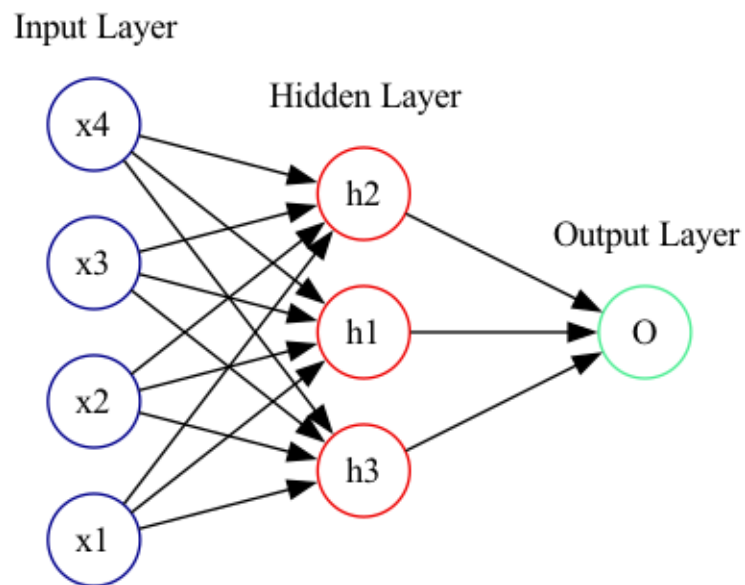
# Fully connect Input → Hidden
inputs = ["x1", "x2", "x3", "x4"]
hidden = ["h1", "h2", "h3"]

for x in inputs:
    for h in hidden:
        nn.edge(x, h)

# Fully connect Hidden → Output
for h in hidden:
    nn.edge(h, "0")

```

```
nn.view()
```



Graphviz automatically generates all connections and organizes the network into layers. This approach is frequently used to visualize Neural Networks; Bayesian Networks; Knowledge Graphs; Computational Graphs

2.5. Saving the Graphs

Graphviz diagrams can be exported into various formats. The most common formats are: PNG; PDF; SVG; JPG.

Saving a graph is straightforward.

```
from graphviz import Digraph

g = Digraph()

g.edge("A", "B")
g.edge("B", "C")

g.render("my_graph", format="png", cleanup=True)
```

This generates: **my_graph.png**

Similarly:

```
g.render("network", format="pdf")
```

creates: **network.pdf**

This makes Graphviz useful for: Reports; Documentation; Research Papers; Presentations; Software Architecture Diagrams

Summary

Graphviz transforms abstract relationships into visual structures that are easy to understand and communicate. In this section, we learned how to:

- Create directed graphs
- Create undirected graphs
- Visualize hierarchical trees
- Customize node colors and styles
- Build neural network diagrams
- Export diagrams into image and document formats

Unlike traditional plotting libraries that focus on numerical data, Graphviz focuses on relationships and connectivity. Whether you are visualizing a social network, software architecture, dependency graph, database schema, decision tree, or neural network, Graphviz provides an elegant way to turn complex structures into clear and meaningful diagrams.

References

<https://graphviz.org/>

<https://sketchviz.com/graphviz-examples>

<https://www.geeksforgeeks.org/deep-learning/how-to-visualize-a-neural-network-in-python-using-graphviz/>

Next Section:

[</> 36. Python: Requests API](#)